For the Guardians of Future

📜 Prologue – The Aether-Forge Shutdown

The **Aether-Forge**, humanity's last autonomous manufacturing facility, requires the **Master Reactivation Sequence (M.R.S.)** to reboot. You, the Auditors, must traverse eight malfunctioning robotic stations. Solving each system's embedded puzzle will yield the next 4-digit code fragment. The mission is to restore the M.R.S. and prevent permanent shutdown.

🗝 *Now Let's go to the Satations Descriptions*

Note: In all station, there must be a way to reset the system and returns it to the initial state/positions, etc.

Table of Contents

Station 1: The Logistics Bot Dispatch - Voice Command Maze Control

📖 Story Overview:

The Automated Logistics Bot (ALB) is stuck and awaiting vocal command authentication. Agents must speak the core activation phrase ("Go Robot!") to unlock the drive system, then use directional voice commands (Forward, Right, Left, Stop) to manually navigate the robot through the physical maze and dispatch its payload.

🔑 Required Clue:

A Voice Command Activation Phrase (e.g., **"Go Robot!"**).

Subsystems:

1. **Initial Voice Activation (Sequential/AI):** Microphone captures and processes the initial authentication phrase.
2. **Voice Command Control (Parallel/AI):** Microphone continuously listens and classifies directional movement commands (Forward, Right, Left, Stop).
3. **Navigation & Drive (Parallel):** Actuates the DC motors based on the received voice command classification.
4. **Reward Release & Confirmation (Sequential):** Detects the robot's completion and actuates the reward.

Feature	Details
Inputs	Microphone Module, Push Button (to initiate listening), Limit Switch
Outputs	DC Motor pair, RGB LED, Servo Motor, Buzzer
AI Twist	**On-device Voice Command Classification** (Keyword Spotting) replaces line-following. The AI state controls activation and continuous steering based on audio input.

🧩 Overall Logic:

- The robot is initially inert. The Agent presses the **Push Button** and speaks the activation phrase: **"Go Robot!"**
- **Calibration/Activation Logic:**
  - The **Microphone Module** captures the audio. The AI model classifies the phrase.
  - If correct, the **RGB LED** turns Green, and the system enters the continuous voice command loop.
  - If incorrect, the **Buzzer** emits three short error pulses.
- **Navigation & Drive Logic:** The Agent uses directional voice commands to move the robot:
  - **"Forward"**: Sets both **DC Motors** to a forward PWM speed (e.g., 150).

  - **"Right"**: Sets the right motor to a reduced speed (e.g., 50) and the left motor to full speed (150) for turning to perform (90°) rotation to right then it stops.
  - **"Left"**: Sets the left motor to a reduced speed (e.g., 50) and the right motor to full speed (150) for turning to perform (90°) rotation to left then it stops.
  - **"Stop"**: Sets both motors to 0 PWM.
- The Agent must provide the commands which will be executed to steer the robot through the physical maze followed by "Stop" wherever the agent wants then providing a new commands (e.g. the agent gives "forward" the robot start to move forward till the agent says "stop" then the robot stops and then a new command is provided).
- **Reward Logic:**
  - Upon reaching the end of the maze, the robot hits the **Limit Switch**. This stops the motors, the **RGB LED** turns Blue, and the success condition is met.
  - The **Servo Motor** rotates 90° for 2 seconds, pushing the contained **RFID Tag** reward into the collection bin.

🎁 Gained Reward:

An **RFID Tag** (containing the 4-digit clue for Station 2, e.g., **2341**).

Station 2: The Component Classifier - Multi-Color Assembly

📖 Story Overview:

The Forge's assembly queue is corrupted. Agents must use the clue (Batch ID) to set the precise counts for three different colored components. A sophisticated conveyor system will then guide and sort blocks, but only if a camera system verifies that the target count for that color hasn't already been reached. Agents must remove blocks if a color stack is already full.

🔑 Required Clue:

The RFID Tag (from Station 1). The tag contains a 4-digit Batch ID (e.g., 2341). This code defines the required counts for the three colors: D_1, D_2, D_3.

Subsystems:

1. **System Activation & Feedback:** RFID reader verifies the Batch ID and initializes the counts. An RGB LED indicates readiness/error.
2. **Block Detection & Transport (Sequential):** Separate sensors manage block placement and arrival at the scan zone.
3. **Sensing & Classification (Parallel/AI):** Reads color (TCS3200) and confirms current stack levels (Camera).
4. **Diverter & Rejection Control (Parallel):** Uses a Servo motor to divert the block or signals rejection with a Buzzer.

Feature	**Details**
Inputs	RFID Reader, TCS3200 Color Sensor (Analog), Limit Switch (Start), Ultrasonic Sensor (Scan Zone), **Camera Module**
Outputs	DC Motor (Conveyor), Servo Motor (Diverter Gate), 16x2 LCD, RGB LED, Buzzer
AI Twist	**Computer Vision Stack Monitoring:** The Camera continuously monitors the three stacking bins to verify the quantity and correct color placement of assembled blocks. This acts as the authoritative counter for the sorting logic.

🧩 Overall Logic:

- **System Activation:** Agent scans the **RFID Tag**. If correct, the system extracts the three component counts ($D_1 = 2, D_2 = 3, D_3 = 4$). **LCD** shows target counts (**R:2, G:3, B:4**) and "ADD BLOCK". **RGB LED** turns Green.
- **Block Placement & Start:**
  - The Agent places a block, activating the **Limit Switch (Start)**.
  - **RGB LED** turns Red. **LCD** shows "BLOCK DETECTED - MOVING." **DC Motor** starts.
- **Scanning Position:**

  - The block arrives at the scan zone, detected by the **Ultrasonic Sensor**. **DC Motor** stops. **LCD** shows "SCANNING..."
  - **Color Sensor (TCS3200)** detects the block's color (Red, Green, or Blue). **LCD** updates (e.g., "BLOCK IS RED").
- **Assembly Check (CV Feedback Loop):**
  - The **Camera Module** classifies the current stack height/count for the detected color (e.g., Red).
  - **Rejection Logic (Stack Full):** If the detected color (e.g., Red) matches a stack that the Camera verifies is **ALREADY FULL** (Red Count = 2), the system enters rejection mode:
    - **Buzzer** emits a continuous tone.
    - **LCD** shows: "COUNT FULL. REMOVE BLOCK!"
    - The **DC Motor** remains stopped and the **Servo Gate** does not move, forcing the Agent to remove the block from the scan zone. The system then resets to "ADD BLOCK."
  - **Acceptance Logic (Sort & Divert):** If the stack is **NOT FULL**:
    - **LCD** shows: "SORTING."
    - The **Servo Motor (Diverter Gate)** rotates to the necessary angle: Right (Red), Forward (Green), or Left (Blue).
    - The **DC Motor** restarts, moves the block past the gate to the appropriate stacking column, and stops.
- **Reward Logic:** The **Camera Module** continuously monitors the stacks. When the Camera confirms all three counts have reached their target values (**R=2, G=3, B=4**), the **RGB LED** turns Blue, the **Buzzer** plays a triumphant jingle, and the reward is displayed.

🎁 Gained Reward:

A 4-Digit Code (e.g., **6348**) displayed on the **LCD**.

Station 3: The Assembly Weaver - Sequential Arm Positioning

📖 Story Overview:

The **4-link 2D robotic arm** requires precise, sequential calibration across four distinct configurations to recalibrate the main assembly matrix. The clue defines the four stages, and in each stage, the Agent must use four control knobs to set the four relative joint angles to a predefined configuration, which is verified by computer vision.

🔑 Required Clue:

A 4-Digit Code (e.g., 6348 from Station 2). This code defines the sequence of four target angles (in 10° steps) that the arm must hold.

Subsystems:

1. **Code Input & Initialization (Sequential):** Keypad validates the code and initializes the four sequential configurations.
2. **Position Control (Parallel/Analog):** User-controlled input dictates the current angle for all four joints simultaneously.
3. **Kinematic Verification (Sequential/AI):** Camera computes the relative joint angles in real-time and compares them to the target configuration.

Feature	**Details**
Inputs	4x4 Keypad, **Potentiometer 1, Potentiometer 2, Potentiometer 3, Potentiometer 4** (Analog), **Camera Module**
Outputs	**Servo Motor 1, Servo Motor 2, Servo Motor 3, Servo Motor 4** (4-link arm), **16x2 LCD**, Buzzer, DC Motor (Gripper/Claw)
AI Twist	**Real-time Computer Vision Kinematic Verification:** The Camera detects colored markers/stickers on each of the four joints, computes the center coordinates, and calculates the **relative angle** between the links. Verification requires all four angles to be within tolerance simultaneously.

🧩 Overall Logic:

- **Code Input & Initialization:** The Agent enters the 4-digit code (**6348**). The system internally loads the four complex target configurations (Config 1-4). The **16x2 LCD** shows the current stage (STAGE 1) and the target angles for all four joints (e.g., $J1 = 45°, J2 = 30°, J3 = 90°, J4 = 0°$).
- **Position Control:** The Agent rotates the **four Potentiometers (Analog)**. The analog 0-1023 readings are mapped to $0° - 180°$ commands for the respective **four Servo Motors** (J1-J4).
- **Kinematic Verification (CV Loop):**
  - The **Camera Module** continuously monitors the robotic arm.
  - The CV algorithm detects the colored markers on the joints to find the (x, y) coordinates of each joint.

  - It then computes the **relative angle** for each joint (θ_1 relative to the base, θ_2 relative to link 1, etc.).
  - The **16x2 LCD** updates the "Current Angles" based on the CV calculation in real-time.
- **Stage Completion:** The system checks if the four computed relative angles are within the target window ($\pm 2°$).
- **Stage 1 Completion:** When all four angles are held within tolerance for a **3-second continuous window**, the **Buzzer** emits a short tone. The system advances, and the **16x2 LCD** shows STAGE 2 and the next target configuration.
- **Stage 2-4:** The Agent repeats the synchronized process for Config 2, 3, and 4.
- **Final Reward Actuation:** After the final configuration (Config 4) is successfully held, the **DC Motor** (Gripper/Claw Actuator) rotates fully to clamp the **QR Code** reward.

🎁 **Gained Reward:**

A unique **QR Code** (printed on the gripped component) containing the 4-digit clue for Station 4 (e.g., **5432**).

Station 4: The Cartesian Assembly Bot - Manual Pick-and-Place

📖 Story Overview:

The Forge's core registry requires precise placement of four unique **metallic Aruco plates** onto a \$5 \times 5\$ grid. The Agent must manually feed the Aruco plates one-by-one into the initial cell \$(1, 1)\$. The **2D Gantry Robot** picks the plate, but the Agent must then act as the motion controller, using **two Potentiometers** to precisely drive the X and Y axes until the **Camera Module** confirms the target placement is achieved. This forms a **manual closed-loop control task** reliant on real-time visual feedback.

🔑 Required Clue:

The **QR Code** (from Station 3). The QR code contains a 4-Digit **Sequence ID** (e.g., **5432**). The agent should scan it with his mobile to open a saved image linked to the QR code saying "your new clue is 5432). This ID directly defines the two target coordinates:

- **Aruco Plate 1 Target:** (X_1, Y_1) (e.g., (5,4))
- **Aruco Plate 2 Target:** (X_2, Y_2) (e.g., (3,2))

Subsystems:

1. **Code Input & Initialization (Sequential):** Keypad or camera scan loads the required coordinate pairs.
2. **Gantry Motion Control (Manual/Analog):** User input from potentiometers dictates the speed/direction of the X and Y DC motors.
3. **Aruco ID/Position Verification (Real-time/AI):** Fixed overhead camera detects the Aruco ID and provides continuous, real-time feedback on the plate's current location.

Feature	Details
Inputs	Camera Module, Push Button (Stage Start), **Potentiometer 1 (X-Axis Control), Potentiometer 2 (Y-Axis Control)**, **2x Limit Switches (X/Y Homing)**
Outputs	**2x DC Motors (X-Axis, Y-Axis)**, **Electromagnet (End Effector)**, **16x2 LCD**, Buzzer, **UV LED/Light**
AI Twist	**Aruco ID Recognition and Continuous Positional Verification:** The Camera tracks the plate, instantly determines if it is **ID 1 or ID 2**, loads the corresponding target coordinate, and provides the live, computed **Current (X, Y) coordinate** to the user via the LCD, enabling manual closed-loop control.

🧩 Overall Logic:

- **Code Input & Initialization:** The Agent scans the **QR Code** to retrieve the Sequence ID (**5432**). The system internally extracts the two target coordinates: **Target 1 = (5, 4)** and **Target 2 = (3, 2)**. The Gantry runs a **Homing Sequence**, establishing the origin. The **16x2 LCD** shows: "HOMING COMPLETE. PLACE EITHER ARUCO at (1, 1)".

- **Stage 1 - Identify & Pick (First Plate):**
  - The Agent places the first metallic Aruco plate at the initial cell (1,1). The Agent presses the **Push Button** to signal readiness.
  - The **Camera Module** detects the **Aruco ID** (either **ID 1** or **ID 2**).
    - *Example:* If **ID 1** is detected, the system loads **Target 1: (5, 4)**.
    - *Example:* If **ID 2** is detected, the system loads **Target 2: (3, 2)**.
  - **LCD** updates: "ID [1/2] DETECTED. T: (X, Y). C: (1, 1)". (E.g., "ID 1 DETECTED. T: (5, 4). C: (1, 1)").
  - The Gantry lowers and the **Electromagnet** is **activated (Magnetized)** to lift the plate. The system internally flags this Aruco ID as 'Placed'.
- **Stage 1 - Manual Motion Control (Closed-Loop):**
  - The two **Potentiometers** are active, controlling the PWM speed and direction of the X and Y **DC Motors**.
  - The **Camera** continuously tracks the Aruco plate being held, calculates its current grid coordinate (**Current X, Y**), and updates the **LCD** in real-time.
  - The Agent uses the **Potentiometers** to manually move the gantry until the **Current (X, Y)** coordinate displayed on the LCD matches the **Target (X, Y)** coordinate.
- **Stage 1 - Verification & Release:**
  - The system checks if **Current** (X, Y) = **Target** (X, Y) for **5 continuous seconds**.
  - If true, the **Electromagnet** is **deactivated (Demagnetized)**, dropping the plate.
  - **Buzzer** emits a confirmation tone. **LCD** updates: "PLACEMENT SUCCESS. ADD REMAINING ARUCO at (1, 1)."
- **Stage 2 - Identify & Pick (Second Plate):**
  - The Agent places the second (remaining) metallic Aruco plate at the initial cell (1,1). The Agent presses the **Push Button** to signal readiness.
  - The **Camera Module** detects the **Aruco ID** (the one not yet 'Placed').
    - *Example:* If **ID 2** is detected, the system loads **Target 2: (3, 2)**.
  - The Gantry lowers and the **Electromagnet** is **activated (Magnetized)**.
- **Stage 2 - Manual Motion Control & Verification:** The Agent repeats the manual control and verification process until **Current** (X, Y) = **Target** (X, Y) for 5 seconds, and the magnet is deactivated.
- **Final Reward Actuation:** Once the **Camera Module** confirms that **both** Aruco plates (ID 1 at (5, 4) and ID 2 at (3, 2)) are in their correct final positions, the **UV LED/Light** activates.

🎁 Gained Reward:

A 4-digit clue (e.g., **4592**) is revealed, written in UV-reactive ink on the 5×5 grid map board.

Station 5: The Remote Ballistics Calibrator - Synced Aim and Fire

📖 Story Overview:

he Forge's critical material injection system requires precise remote calibration before firing. Agents must remotely enter the 4-digit calibration code to set the target distance and the required number of successful hits. The challenge relies on a gravity-fed ball drop system, where the Agent uses a local **Potentiometer** to synchronize the horizontal aiming angle of the launcher (via the web interface) before pressing a remote **"SHOOT"** button to physically release the ball.

🔑 Required Clue:

A 4-Digit Code (e.g., **4592** from Station 4). This code defines two critical factors, using only the first two digits:

- **Target Tunnel Index (1-7):** D_1 (e.g., **4**). This digit sets the target distance to the end of the specified tunnel guide.
- *Mapping Example:* Tunnel 1 = 15 cm, Tunnel 4 = 40 cm, Tunnel 7 = 75 cm.
- **Required Successful Hits (1-9):** D_2 (e.g., **5** hits).

Subsystems:

1. **Network Access & Input (Sequential):** Hosts a web server and receives the 4-digit clue and fire commands.
2. **Target Positioning (Sequential):** DC motor moves the target plate to the required distance at the end of the correct guided tunnel.
3. **Aim Control Synchronization (Parallel):** Reads the physical potentiometer and sends the current angle via WebSockets to the client for synchronization.
4. **Launcher Actuation & Feedback (Sequential):** Servo 2 acts as a mechanical gate for gravity launch; Limit Switch detects a successful hit.

Feature	Details
Inputs	Pico W Web Server (HTTP POST, Wi-Fi), **Potentiometer** (Analog - Aiming Control), **Limit Switch** (Target Hit Sensor), **Ultrasonic Sensor** (Target Distance Verification)
Outputs	**Servo Motor 1** (Launcher Rotation/Aim) with fork-like end effector, **Servo Motor 2** (Ball Release Gate/Tipping Fork), **DC Motor** (Linear Stage/Target Distance), **16x2 LCD**, WebSockets (Real-time Angle Sync), Buzzer, RGB LED
AI Twist	**Web-Synched Analog Position Control:** The physical **Potentiometer** directly sets the positional angle of **Servo Motor 1**. This actual angle is read by the Pico W and pushed to the web interface via WebSockets to visually synchronize the aiming arrow, giving the user critical remote feedback for the firing decision.

🧩 Overall Logic:

- **Physical Setup:** The ball launcher platform is mounted on an incline. **Servo Motor 1 (Aim)** controls the horizontal angle of the launcher assembly, using the potentiometer for direct positional input. **Servo Motor 2 (Gate)** holds the ball at 0° and releases it at 90°.
- **System Activation & Clue Entry:**
  - The Agent presses a **Start Button**. The **16x2 LCD** displays the Pico W's IP address and a prompt: "ACCESS WEB SERVER."
  - The Agent connects to the web interface and enters the 4-digit clue (**4592**) on the digital keypad and presses **"ENTER CLUE"**.
- **Target Setup & Positioning:**
  - The system extracts the values: Tunnel Index **4** (D_1) and Required Hits **5** (D_2).
  - **Tunnel Position:** The system converts Index 4 to the corresponding distance (e.g., 40 cm). The **DC Motor** drives the **Linear Stage** until the **Ultrasonic Sensor** reads this precise position.
  - **Distance Verification (Ultrasonic):** The **Ultrasonic Sensor** continuously takes readings. When the reading is within tolerance (± 1 cm) of the target distance (e.g., 40 cm), the **DC Motor** stops.
  - **LCD** updates: "TUNNEL SET: 4 (40 cm). HITS REQ: 5." The web interface reflects this information and is now ready for aiming.
- **Aiming Synchronization (Real-time Servo Control):**
  - The Agent uses the physical **Potentiometer** near the station. The potentiometer's analog value is read by the Pico W, mapped to the $0^\circ - 180^\circ$ **range**, and sent as the command to **Servo Motor 1 (Aim)**.
  - The physical **Servo Motor 1** moves continuously as the Agent adjusts the potentiometer.
  - The Pico W simultaneously sends this current servo angle (e.g., 45.2°) via **WebSockets** to the web interface.
  - The web interface displays an **Arrow** that rotates in real-time, mirroring the physical launcher angle. The Agent must precisely adjust the potentiometer until the arrow aligns with the target tunnel on the web.
- **Shooting & Verification (Gravity Launch):**
  - When the Agent is confident in the aim, they press the **"SHOOT" button** on the web interface.
  - This sends a command to the Pico W. The system checks the current angle of **Servo Motor 1**. If the angle is correctly aligned with the target tunnel ($\pm 1^\circ$):
    - **Servo Motor 2 (Gate)** moves to 90° (open), releasing the ball.
    - **Servo Motor 2** returns to 0° (closed) to block the next ball.
  - If the ball hits the target, the **Limit Switch** is momentarily pressed.

    - The **Buzzer** emits a short chime, and the successful hit count increases **on the web interface only**.
- **Stage Completion:**
    - The Agent must successfully hit the target **5 times** (the value of D_2).
    - Upon the 5th successful hit, the **RGB LED** turns Blue, the **Buzzer** plays a triumphant jingle, and the web interface displays the next clue.

🎁 **Gained Reward:**

The next 4-digit clue (e.g., **2143**) displayed on the remote web interface.

Station 6: The Bio-Sensor Override - Multi-Axis Gesture Authentication

📖 Story Overview:

The system requires a biometric key and precise robotic calibration over two stages. Agents must first present the required number of fingers for a camera scan and confirm the system's reading. They must then maintain a **critical distance** from the station while the 2-link robotic arm **automatically** moves to the target angles computed from the confirmed finger count. This creates a synchronization challenge requiring the Agent to stabilize their position while the system stabilizes its mechanical position.

🔑 Required Clue:

A 4-Digit Code (e.g., **2143** from Station 5). This code defines the required sequence of two finger counts and the required distance:

- **Sequence of Finger Counts:** D_1, D_2 (e.g., **2, 1**).
- **Required Agent Distance:** $D_3 \times 20$ cm (e.g., 4×20 cm $= 80$ cm).

Subsystems:

1. **Code Input & Stage Initialization (Sequential):** Keypad validates the code and initializes the two-stage authentication process.
2. **Gesture Scanning & User Confirmation (Sequential/AI):** Camera captures the hand, and the user confirms the CV reading.
3. **Arm Movement Control (Sequential/Automatic):** Servos automatically move to the target angles.
4. **Distance & Kinematic Verification (Parallel/AI):** Ultrasonic Sensor checks distance; Camera computes arm's angle.

Feature	Details
Inputs	4x4 Keypad, **Camera Module (Overhead)**, **Ultrasonic Sensor (Distance)**, **'Correct' Push Button, 'Wrong' Push Button**
Outputs	**Servo Motor 1, Servo Motor 2** (2-joint horizontal arm), **16x2 LCD** (Display for all instructions and feedback), **DC Motor** (Pulley/String mechanism for final clue door)
AI Twist	**Two-Phase CV Integration:** The Camera performs **Phase 1 (Finger Count)** to set targets, and then continuously performs **Phase 2 (Arm Kinematics)**. For Phase 2, the Camera detects the joint markers, computes the coordinates, and calculates the **relative angles** (C_1, C_2) between the links in real-time.

🧩 Overall Logic:

- **Code Input & Initialization:** Agent enters the 4-digit clue (**2143**).
    - Finger Counts: **Count 1 = 2** (D_1) and **Count 2 = 1** (D_2).
    - Target Distance: 80 cm ($D_3 \times 20$ cm).
- **Stage Target Calculation:** For each stage, the system calculates the two target joint angles:

  - **Servo 1 Target Angle:** $\times 10°$
  - **Servo 2 Target Angle:** $\times 20°$
  - *Example: Stage 1 (Count 2) Target Angles:* $2 \times 10° = 20°$ *and* $2 \times 20° = 40°$.
- **Stage 1 - Phase 1: Finger Scan (Count 2):**
  - **LCD** displays: STAGE 1 | TGT: 2 | RAISE FINGERS FOR 5 SEC.
  - Agent raises fingers beneath the **Overhead Camera**.
  - After 5 seconds, the **Camera Module** classifies the number of extended fingers (X).
  - **LCD** updates: DETECTED: X FINGERS | PRESS CORRECT OR WRONG.
  - If the Agent presses **'Wrong'** or the detected count $X \neq 2$, the system repeats the scan (loop back).
  - If the Agent presses **'Correct'** and $X = 2$, the system proceeds to Phase 2.
- **Stage 1 - Phase 2: Arm Calibration (20°/40° - Distance 80cm):**
  - **LCD** displays: TGT DIST: 80 CM | PLEASE STEP BACK.
  - The system waits until the **Ultrasonic Sensor** detects the Agent within the target range (e.g., 80 ± 5 cm).
  - Once the correct distance is maintained for 1 second, the **Servos** begin a slow, controlled, **automatic** movement toward the target angles ($20^{\circ} and 40^{\circ}$).
  - This should be while maintining the agent target distance.
  - **LCD** updates continuously: C_DIST: 82 CM | C1: X.X C2: Y.Y.
  - **CV Angle Calculation (Phase 2 Detail):** During arm movement, the **Camera Module** is tracking **colored markers** or specific visual features located on the two robotic links. The Computer Vision (CV) algorithm uses the (x, y) coordinates of the base joint, joint 1, and the end effector to dynamically calculate the two relative angles: $\boldsymbol{C_1}$ (relative to the base) and $\boldsymbol{C_2}$ (relative to Link 1). The LCD displays these real-time, calculated angles (C1: X.X C2: Y.Y) used for the Kinematic Check.
- **Stage 1 - Verification & Completion:**
  - **Parallel Check:** The system must satisfy two conditions simultaneously for **3 continuous seconds**:
    - **Distance Check: Ultrasonic Sensor** reading is within tolerance of 80 cm.
    - **Kinematic Check: Camera Module** confirms C1 and C2 are within tolerance ($\pm 2°$) of the targets $(20°, 40°)$.
  - If conditions are met, the **Buzzer** emits a ready tone.
  - **LCD** prompts: TARGET REACHED! | CONFIRM (CORRECT BUTTON).
  - Agent presses the **'Correct' Button**. Stage 1 is complete.

- **Stage 2 Execution (Count 1):** The Agent repeats Phase 1 and Phase 2 for **Count 2** (e.g., **1** finger). Target angles: $1 \times 10^\circ = 10^\circ$ and $1 \times 20^\circ = 20^\circ$. The distance requirement remains 80 cm.
- **Final Reward Actuation:** After Stage 2 is successfully completed and confirmed, the **DC Motor** rotates the attached pulley, and the string lifts the door to reveal the final clue.

🎁 **Gained Reward:**

The next 4-Digit Code (e.g., **4915**) is revealed behind the door.

Station 7: The Logic Grid - Remote Mobility Challenge

📖 Story Overview:

The **Logic Grid** manages the Forge's remote maintenance drones. To prove access, Agents must pilot a small **mobile robot** through a physical maze using a **web interface**. This control is highly sensitive and requires continuous bio-authentication: the Agent must have their hand on the **Capacitive Orb** to enable movement commands. Upon solving the physical maze, the robot reveals the final 4-digit code fragment, which is then used in a mathematical riddle to get the final activation sequence digit.

🔑 Required Clue:

A 4-Digit Code (e.g., **4915** from Station 6).

Subsystems:

1. **Code Validation & UI Initialization (Sequential):** The virtual Keypad validates the 4-digit clue code and, upon success, initializes the visual maze, virtual robot icon, and directional controls.
2. **Bio-Authentication Gating (Parallel/Feedback):** The **Capacitive Orb Sensor** continuously monitors user presence to enable/disable movement commands and provides immediate feedback via the **RGB LED** and **Buzzer**.
3. **Command Execution & Synchronization (Sequential/Synchronized):** Receives the user's directional input, verifies the bio-authentication state, and executes a single, synchronized step command for both the **physical robot** (via H-Bridge/Motors) and the **virtual robot display**.
4. **Endgame Verification & Reward Release (Mechanical/Automatic):** The **Limit Switch** at the final cell confirms the physical robot's successful traverse, automatically triggers the **Electromagnet** deactivation, and controls the final robot movement to reveal the clue.

Feature	Details
Inputs	**Keypad UI (Virtual)** for 4-digit code authentication; **Capacitive Orb Sensor** for bio-authentication; **Web Directional Commands** for robot movement; **Limit Switch** to confirm maze completion.
Outputs	2 x **DC Geared Motors** (via H-Bridge) for robot drive; **Electromagnet** to release the final clue; **RGB LED** for Orb status; **Buzzer** for auditory warnings.
AI Twist	The **Web Interface** is the twist, acting as the intelligent control layer. It performs **gating logic** (input ignored unless bio-authenticated) and maintains **perfect synchronization** between the abstract virtual grid state and the precise physical stepper/motor commands sent to the robot.

🧩 Overall Logic:

- **Phase 1: Clue Authentication & Maze Load**

- The Web Interface initially displays only the **4-Digit Keypad UI** and a **START** button. The maze and directional controls are **hidden**.
- The Agent must enter the 4-digit code clue (e.g., **4915**) using the Keypad UI.
- Pressing **START** validates the code against the system's required value.
  - **SUCCESS:** The **virtual maze grid**, **virtual robot icon**, and the **four directional arrow controls** instantly **appear** on the Web Interface. The system moves to Phase 2.
  - **FAILURE:** An error message is displayed: "CODE INVALID: ACCESS DENIED."

- **Phase 2: Remote Mobility Challenge & Authentication Feedback**
  - **Orb Status Feedback:**
    - **Capacitive Orb Active (Hand Placed):** The **RGB LED turns GREEN**. The system is ready to receive movement commands.
    - **Capacitive Orb Inactive (Hand Removed):** The **RGB LED turns RED**. The **Buzzer is activated for 3 seconds** as a warning tone, informing the user they have lost authentication.
  - **Movement Gating & Synchronization:**
    - When an Agent presses a directional arrow (UP/DOWN/LEFT/RIGHT), the system **checks the Orb Sensor**.
    - **IF** the Orb is active: The command is sent to the mobile robot, causing it to move **one step**. Simultaneously, the **virtual robot icon** on the Web Interface moves **one corresponding step**.
    - **IF** the Orb is inactive: The button press is ignored, and the web interface displays: "AUTHENTICATION FAILED: PLACE HAND ON ORB TO ACTIVATE."
  - The Agent navigates the mobile robot through the physical maze grid.
- **Phase 3: Reward Release & Riddle**
  - The **Limit Switch** is triggered by the robot reaching the 'END' cell.
  - The Pico W sends a signal to **deactivate the electromagnet**, dropping the metallic reward plate (e.g., printed with **4567**).
  - The robot executes a final command: move **one step back** to reveal the dropped plate.
  - The Web Interface updates, displaying the final riddle:
    - "The product of the first two digits plus the sum of the last two: $(D_1 \times D_2) + (D_3 \times D_4)$"

🎁 **Gained Reward:**

A mathematical equation or riddle displayed on the **Web Interface** (e.g., "The product of the first two digits plus the sum of the last two: $(D_1 \times D_2) + (D_3 \times D_4)$"). The solution is **62**.

Station 8: The Master Reactivation Sequence - Physical Tic-Tac-Toe Showdown

📖 Story Overview:

The final, 28-digit **Master Reactivation Sequence (M.R.S.)** must be authenticated to boot the Aether-Forge. This is achieved by proving the Auditors' capacity for both strategy and physical precision. Agents must play a decisive, physical 3×3 Tic-Tac-Toe game against the formidable **2D Cartesian Gantry Robot**.

- **Agent (O):** Player's pieces (non-metallic) placed by hand.
- **System (X):** Robot's pieces (metallic 'X' plates) placed by the gantry/electromagnet.

The **System (X)** is running the **Minimax Algorithm** for optimal, perfect play. The Agent **must win** the game by placing their 'O' piece on the final winning square, or the system will reset, and the 28-digit code must be re-entered.

🔑 Required Clue:

The **Master Reactivation Sequence (M.R.S.) Key** (6 Digits) as follows:

C1 → D4 statio 1 reward
C2,3 → D3,4 station 4 reward
C4 → D4 station 5 reward
C5,6 → D4 station 7 equation solution

- **M.R.S. Example Key: 192362**

Subsystems:

- **M.R.S. Key Input (Sequential):** Keypad requires the full 6-digit code before system activation.
- **Board State Monitoring (Real-time/AI):** Camera detects 'O' (player) and 'X' (robot) positions.
- **Robot Movement Control (Sequential/AI):** Gantry/Electromagnet subsystem executes the Minimax-calculated move.

Feature	Details
Inputs	**4x4 Keypad** (for 6 digits), **Overhead Camera Module**, Push Button ('Confirm Move')
Outputs	**2x DC Motors (X-Axis, Y-Axis)**, **Electromagnet (End Effector)**, **16x2 LCD** (Instructions/Status), Buzzer, **Massive Reward Actuator**
AI Twist	**Physical Minimax Player:** The Gantry Robot is the physical manifestation of the Minimax AI. The Camera detects the player's 'O' move, the Pico W calculates the optimal 'X' counter-move via Minimax, and the robot physically executes the move using the electromagnet to pick up and drop the metallic 'X' plate.

Overall Logic:

- **M.R.S. Key Input & Activation:**
  - **LCD** displays: "ENTER M.R.S. (6 DIGITS)".
  - The Agent enters the 28-digit sequence via the **Keypad**.
  - If correct, the **LCD** displays: "KEY ACCEPTED. AETHER-FORGE STANDBY. START GAME (PUSH BUTTON)."
  - If incorrect, the **Buzzer** emits a long error tone, and the system resets.
- **Game Start:**
  - The Agent presses the **Push Button** to start the game.
  - The **Gantry Robot** runs a **Homing Sequence** (returning to a safe X-piece 'stockpile' area).
  - **LCD** displays: "AGENT (O) GOES FIRST."
- **Player's Move (O):**
  - The Agent manually places an 'O' piece on an empty square.
  - The Agent presses the **Push Button** to signal their move is complete.
- **AI Turn (X):**
  - The **Camera Module** scans the board and identifies the position of the newly placed 'O' piece.
  - The Pico W (System X) runs the **Minimax Algorithm** to calculate the optimal counter-move (X) to win or force a draw.
  - **LCD** displays: "SYSTEM (X) CALCULATING..."
  - The **Gantry Robot** moves from the stockpile area to the calculated square.
  - The **Electromagnet** is **activated** to lift an 'X' piece from the stockpile.
  - The robot moves to the target square, lowers the piece, and the **Electromagnet** is **deactivated** to release the metallic 'X' plate.
  - **LCD** updates: "SYSTEM (X) MOVED."
- **Game Continuation:** The game alternates until a win or draw is detected.
- **Final Actuation Condition:**
  - If the **Agent (O) wins** (Camera detects three 'O's in a row):
    - **LCD** displays: "AETHER-FORGE REACTIVATED. MISSION SUCCESS."
  - If the **System (X) wins** or a **Draw** occurs:
    - **Buzzer** emits a series of rapid failure tones.
    - **LCD** displays: "M.R.S. FAILED. RE-ENTER KEY." The system returns to the 28-digit input state.

Gained Reward:

The activation of the **DC Motor** and linear stage, delivering the final **Master Reactivation Sequence (M.R.S.)** envelope.

